Tecnicatura universitaria en programación

TRABAJO PRACTICO INTEGRADOR

FOOD STORE

Programación 2

_Gonzalo Carmona

_UTN universidad nacional tecnologica

_Entrega: 16/11

Índice

1. Introducción y objetivos pag 3
2. Marco teórico pag 3
3. Modelo y relaciones pag 4
4. Arquitectura y decisiones técnicas pag 4
5. Reglas de negocio y validaciones pag 5
6. Pruebas pag 5
7. Dificultades y soluciones pag 5
8. Bibliografía y enlaces pag 6

1. Introducción y objetivos

Food Store es una aplicación de consola desarrollada en Python para gestionar categorías, productos, usuarios y pedidos de un negocio de comidas. La información se mantiene en memoria durante la ejecución; no se utiliza una base de datos, de acuerdo con el alcance de la consigna. El objetivo es aplicar programación orientada a objetos mediante clases, encapsulamiento, herencia, abstracción, polimorfismo, composición, enumerados, colecciones, excepciones y anotaciones de tipos.

2. Marco teórico

Programación orientada a objetos: organiza el sistema en objetos que reúnen estado y comportamiento. En este proyecto, Categoria, Producto, Usuario y Pedido representan conceptos del negocio.

Encapsulamiento: los atributos se mantienen privados y se accede a ellos mediante propiedades. Los setters validan valores antes de modificar el estado.

Herencia: las entidades heredan de Base, que concentra identificador, marca de baja lógica, fecha de creación e igualdad por id.

Clase abstracta e interfaz: Base no se instancia directamente. Calculable define el contrato calcular_total(), implementado por Pedido.

Polimorfismo: Menu es abstracta y MenuAdministrador y MenuCliente implementan sus propios menús. main solo solicita mostrar el menú creado.

Asociación, agregación y composición: Producto referencia una Categoria; los servicios administran las colecciones. Pedido crea y contiene sus DetallePedido, que forman parte de su ciclo de vida.

Enumerados: Rol, Estado y FormaPago restringen los valores posibles a opciones definidas.

Excepciones: los errores del dominio derivan de FoodStoreError para que la consola pueda comunicarlos sin cerrar inesperadamente.

Type hints: las anotaciones de tipos documentan parámetros, atributos y retornos, facilitando lectura y mantenimiento.

3. Modelo y relaciones

Base contiene id, eliminado y creado_en. Categoria organiza productos; Producto conserva una referencia a su categoría. Usuario contiene datos de acceso, sin exponer la contraseña mediante una propiedad. Pedido se vincula con un usuario y contiene detalles. Cada DetallePedido referencia un producto, cantidad y subtotal.

El identificador se genera con un contador compartido en Base. La igualdad de las entidades se determina por id. Las fechas se asignan automáticamente en el constructor.

4. Arquitectura y decisiones técnicas

El proyecto separa responsabilidades en tres capas: entidades, servicios y consola. Las entidades validan su propio estado; los servicios administran colecciones y reglas de negocio; la consola solicita datos y presenta mensajes.

main.py instancia servicios, carga datos iniciales y administra el inicio de sesión. La sesión se mantiene en una variable local y se pasa al menú correspondiente. El almacenamiento es temporal y no persiste al cerrar el programa.

Las bajas son lógicas: el objeto permanece en la colección, pero se excluye de listados y operaciones nuevas.

5. Reglas de negocio y validaciones

El precio y el stock no pueden ser negativos; las cantidades de detalle deben ser mayores que cero; el mail es único; los nombres de categoría no se repiten sin distinguir mayúsculas; y no se puede eliminar una categoría que tenga productos activos.

El inicio de sesión permite tres intentos y muestra un mensaje genérico cuando las credenciales son incorrectas. El cliente dispone únicamente de consultas y puede acceder solo a sus propios pedidos.

Los estados siguen las transiciones PENDIENTE -> CONFIRMADO -> TERMINADO y PENDIENTE/CONFIRMADO -> CANCELADO. Al cancelar o eliminar un pedido no terminado, se devuelve el stock.

Antes de crear un pedido, el servicio valida los productos y cantidades. Solo después de validar todo agrega los detalles, descuenta el stock y guarda el pedido, evitando cambios parciales cuando falta stock.

6. Pruebas

El archivo pruebas.py incluye pruebas básicas del modelo, unicidad de mail, creación de pedido con descuento de stock, rechazo por stock insuficiente sin alterar los datos y rechazo de una transición de estado inválida.

Para ejecutarlas se utiliza el comando python pruebas.py desde la raíz del proyecto. La aplicación se ejecuta con python main.py.

7. Dificultades y soluciones

Una dificultad central consiste en coordinar las relaciones entre objetos y las reglas de negocio. Para evitar duplicación de lógica, las colecciones y las búsquedas se centralizan en servicios.

Otro caso importante es la falta de stock al registrar pedidos. Se resuelve validando todas las cantidades antes de modificar el stock o incorporar el pedido a la colección.

Las excepciones propias permiten diferenciar problemas de validación, duplicados, entidades inexistentes y operaciones no permitidas, mientras que la consola captura esos errores y muestra mensajes entendibles.

8. Bibliografía y enlaces

Python Software Foundation. Documentación oficial de Python: https://docs.python.org/3/ Python Software Foundation. Módulo abc: https://docs.python.org/3/library/abc.html Python Software Foundation. Módulo enum: https://docs.python.org/3/library/enum.html

Recurso de ayuda: https://chatgpt.com/

Repositorio GitHub: gonzzacar/Food_store: Sistema de gestión de pedidos de comida desarrollado en Python para el trabajo práctico integrador de Programación II.

Video del trabajo: https://youtu.be/4-hDkXY_Z5A?si=bHUyJNYMtnAPXF5L